

1. DÉFINITION ET IMPLÉMENTATION

On veut représenter des partitions d'un ensemble fini $\llbracket 0, n - 1 \rrbracket$. Sur une telle partition, on veut pouvoir réaliser efficacement les deux opérations suivantes : déterminer si deux éléments sont dans la même classe de la partition, et modifier la partition pour réunir deux classes en une seule.

Pour chaque ensemble de la partition, on choisit arbitrairement un entier de cet ensemble qui joue le rôle de *représentant*.

La partition est représentée par un tableau d'entiers `classes` qui à chaque entier k associe :

- soit une valeur strictement négative, qui signifie que k est le représentant de sa classe ;
- soit une valeur positive u , ce qui signifie que pour trouver le représentant de k il faut déterminer le représentant de u .

Question 1. On considère le tableau `classes` suivant :

```
let classes = [| -1; -3; 1; 4; -2; 1; 2; 6 |]
```

OCAML

Déterminer le nombre de classes de la partition et le représentant de chaque classe.

On veut écrire une fonction `uf_find` qui à partir d'un entier v et du tableau des classes renvoie le représentant de la classe qui contient v .

Cette fonction utilise une optimisation, dite de *compression de chemins* : lors du premier appel à `find classes v`, il faut remonter d'entier en entier jusqu'à trouver le représentant u . On profite de cette remontée pour *modifier* le tableau `classes` en posant, une fois le représentant connu, `classes.(v) ← u`. Ainsi, l'appel suivant `uf_find` pour l'entier v renverra (presque) directement la valeur u .

Avec l'exemple précédent, si on appelle `uf_find classes 7`, alors le tableau vaudra à la fin de l'exécution :

```
[| -1; -3; 1; 4; -2; 1; 1; 1 |]
```

OCAML

Question 2. Écrire la fonction `uf_find`.

```
val uf_find : int array -> int
```

OCAML

Question 3. En déduire une fonction `uf_meme_classe` qui décide si deux entiers sont dans la même classe.

```
val uf_meme_classe : int array -> int -> int -> bool
```

OCAML

Étant donné deux entiers u et v , on veut à présent écrire une fonction `uf_union` qui fusionne les deux classes contenant les entiers u et v . On commence par déterminer les représentants respectifs r_u et r_v de u et v et on pose au choix `classes.(ru) ← rv` ou bien `classes.(rv) ← ru`.

Pour choisir quelle modification apporter, on utilise une optimisation dite de *rang*. Le *rang* d'un représentant r est la valeur de `classes.(r)`. Cette valeur est par définition positive. Il s'agit d'un majorant du nombre de sauts nécessaires pour trouver r à partir d'un entier quelconque de la classe.

Au début, tous les rangs valent 1. Lorsque l'on fusionne les classes associées à deux représentants r_u et r_v alors :

- on pose `classes.(rv) ← ru` lorsque le rang de r_u est strictement plus grand que celui de r_v ;
- on pose `classes.(ru) ← rv` lorsque le rang de r_v est strictement plus grand que celui de r_u ;
- en cas d'égalité de rang, on choisit arbitrairement `classes.(rv) ← ru` et on augmente le rang de r_u de 1 (ou le contraire, en échangeant `rv` et `ru`, c'est arbitraire).

Question 4. Écrire la fonction `uf_union`.

```
val uf_union : int array -> int -> int -> unit
```

OCAML

Question 5. Écrire une fonction `uf_new` qui initialise un tableau de classes où toutes les classes sont un singleton.

```
val uf_new : int -> int array
```

OCAML

2. ÉTUDE DE LA COMPLEXITÉ

On part d'une partition formée uniquement de singletons, dont les rangs initiaux valent 1. On réalise une succession de réunions.

Question 6. Montrer que, si k est un représentant de rang r_k , alors le nombre d'éléments de la classe de k est au moins 2^{r_k-1} .

Question 7. Montrer que `uf_union` et `uf_find` sont en $O(\log(n))$.

En réalité, ces opérations sont bien plus rapides, mais la preuve est plus compliquée.

3. FABRICATION DE LABYRINTHES

On dispose d'une grille formée de cases hexagonales, avec n lignes d'hexagones et p hexagones par ligne. On veut créer un labyrinthe aléatoirement tel que pour tout couple de cases, il existe un unique chemin reliant ces deux cases. Pour cela, on part de la liste des murs intérieurs (les murs sur le périmètre de la grille sont exclus), on mélange cette liste. On considère ensuite les murs mélangés un par un, et on abat un mur si et seulement s'il se trouve entre deux cases qui n'étaient pas reliées auparavant.

Pour désigner les cases hexagonales, on choisira un système de coordonnées adapté, par exemple parmi les classiques présentés par <https://www.redblobgames.com/grids/hexagons/>.

Question 8. Écrire une fonction `voisines` qui à partir des coordonnées d'une case et des dimensions de la grille donne les coordonnées des cases voisines.

Question 9. Écrire une fonction `murs` qui, à partir des dimensions de la grille, donne la liste de tous les murs. Chaque mur est représenté par le couple de cases qu'il sépare.

Question 10. Combien de murs possède la grille à n lignes et p hexagones par ligne ?

On va mélanger les murs. En OCaml, on a un générateur de nombres aléatoires dans le module `Random`. Ce générateur doit être initialisé avec un premier appel à `Random.self_init ()`. On obtient ensuite un entier aléatoire dans $\llbracket 0, a \rrbracket$ avec un appel à `Random.int a`.

Question 11. Proposer une fonction `mélange_murs` qui prend en paramètre le tableau des murs `murs` et sa longueur, et qui mélange en place les murs.

Question 12. Ce mélange est-il uniforme parmi toutes les permutations possibles ?

Question 13. Générer un labyrinthe, représenté par la liste des murs *présents*.

On peut produire facilement une sortie graphique du labyrinthe en générant une image au format SVG. Une image SVG est un document XML qui a la structure suivante :

XML

```

<svg version="1.1"
  width="300" height="200"
  xmlns="http://www.w3.org/2000/svg">

  <!-- Dessin d'une ligne entre deux points
  L'origine du repère est en haut à gauche, l'axe x est horizontal vers
  la droite, l'axe y est vertical vers le bas.
  -->
  <line x1="10" x2="50" y1="110" y2="150" stroke="black" stroke-width="3"/>
</svg>

```

Si vous voulez en apprendre plus sur SVG, la documentation de MDN est très bien faite, en particulier : https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorials/SVG_from_scratch

Question 14. Pourriez-vous dessiner ce labyrinthe ? On rappelle quelques fonctions utiles en OCaml :

```

(** Ouverture d'un fichier en écriture *)
val open_out : string -> out_channel
(** Fermeture d'un fichier *)
val close_out : out_channel -> unit
(** Un fprintf qui fonctionne comme en C *)
val Printf.fprintf : out_channel -> ('a, out_channel, unit) format -> 'a

```

OCAML

4. FAIRE LA COURSE

En OCaml, on peut mesurer le temps processeur utilisé par le programme actuel grâce à la fonction `Sys.time`. Elle renvoie un flottant qui estime le temps processeur utilisé par le programme depuis le début de son exécution.

Question 15. Écrire un programme qui prend en paramètres, sur la ligne de commande, deux entiers n_{min} et n_{max} . Pour chaque entier n entre ces deux bornes, il génère une instance d'unir et trouver de taille n , puis réalise aléatoirement n fusions (ce qui, parfois ne fait rien sur des sommets déjà fusionnés). En sortie, votre programme produit un fichier au format CSV qui donne le temps d'exécution (moyen) pour chaque valeur de n :

```

n,millis
42,245
43,257
500,2104

```

CSV

Question 16. Tracer les courbes. Si vous êtes sages et efficaces, je vous montre gnuplot. Comparer les temps d'exécution en fonction de l'utilisation, ou non, des optimisations.